

Agent System Management

Managing & Observing Agentic Systems

As agentic systems grow in complexity, ensuring **reliability, safety, and clarity** becomes just as important as building core logic.

Multi-agent workflows can quickly become hard to reason about due to numerous agents, tools, and handoffs interacting dynamically.

To address this, the **OpenAI Agents SDK** provides built-in capabilities for **visualization, guardrails, observability, and testing**.

Learning outcome

- Agent Visualization**

- Generate graphical diagrams of agents, tools, and interactions
- Improve system clarity, debugging, and architecture understanding

- Guardrails**

- Implement input and output validations, Prevent unsafe, irrelevant, or policy-violating content

- Logging, Tracing & Observability**

- Use the Traces module to track:
 - Model calls, Tool executions, Agent handoffs, Guardrail triggers
- Add custom traces and spans for deeper insights

- Agent Testing**

- Apply unit and end-to-end testing strategies
- Validate reliability despite non-deterministic agent behavior

Agent visualization

- create the visualization graphs, we must first install the required dependencies. In your terminal, activate your environment, and then run the following command, which installs the dependency
- `$ pip install "openai-agents[viz]"`

```
from agents import Agent, Runner, SQLiteSession, trace, function_tool
from agents.extensions.visualization import draw_graph
```

```
# Create tools
```

```
@function_tool
```

```
def calculate_physics_equation(equation):
    pass
```

```
@function_tool
```

```
def perform_culture_survey(goal):
    pass
```

```
# Create our agents

# Specialized science agents
physics_agent = Agent(name="Physics Agent", instructions="Answer questions about physics.", tools=[calculate_physics_equation])
chemistry_agent = Agent(name="Chemistry Agent", instructions="Answer questions about chemistry.")
medical_agent = Agent(name="Medical Agent", instructions="Answer questions about medical science.")

# Specialized history agents
politics_agent = Agent(name="Politics Agent", instructions="Answer questions about political history.")
warfare_agent = Agent(name="Warfare Agent", instructions="Answer questions about wars and military history.")
culture_agent = Agent(name="Culture Agent", instructions="Answer questions about cultural history.", tools=[perform_culture_survey])
```

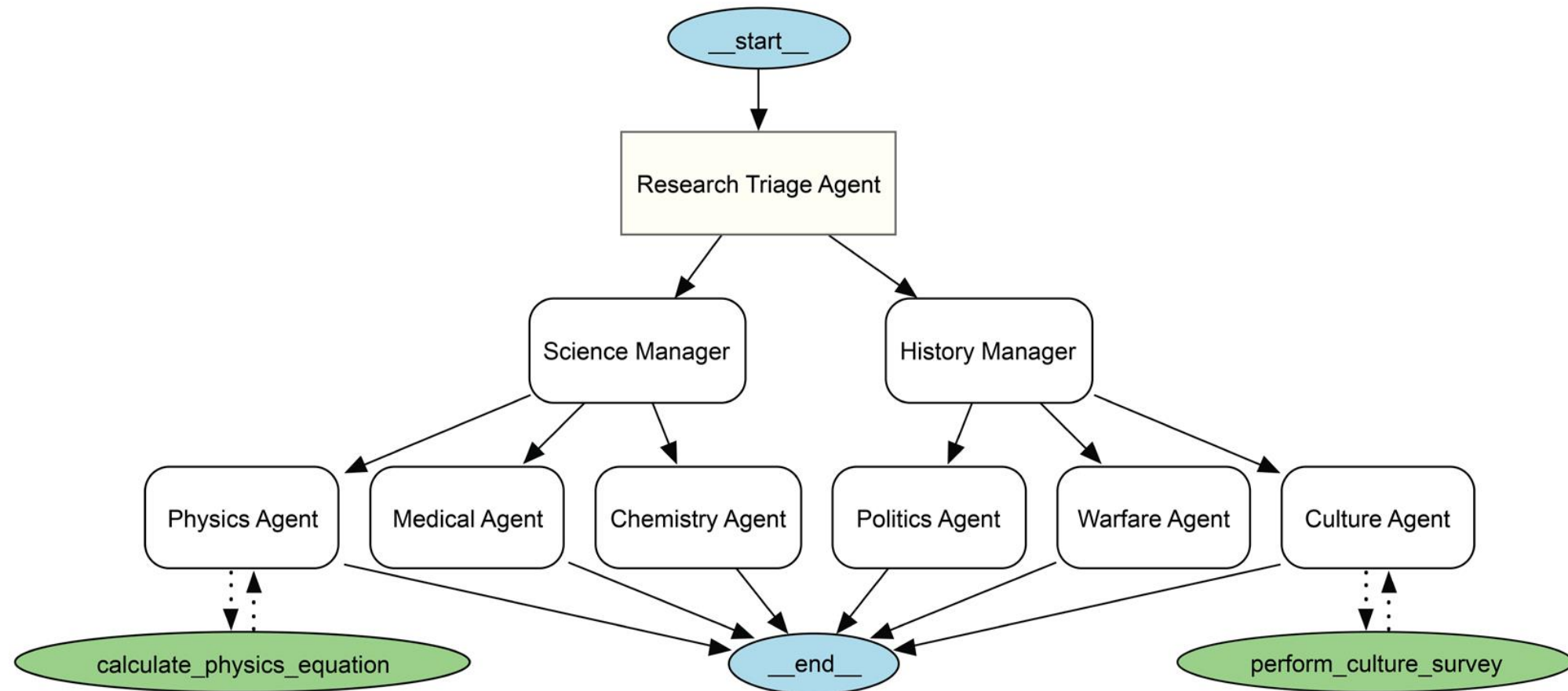
```
# Manager agents with handoffs to their respective domains
science_manager = Agent(
    name="Science Manager",
    instructions="Manage science-related queries and route them to the
appropriate subdomain agent.",
    handoffs=[physics_agent, chemistry_agent, medical_agent]
)

history_manager = Agent(
    name="History Manager",
    instructions="Manage history-related queries and route them to the
appropriate subdomain agent.",
    handoffs=[politics_agent, warfare_agent, culture_agent]
)
```

```
# Top-level triage agent
triage_agent = Agent(
    name="Research Triage Agent",
    instructions="Triage the user's question and decide whether it's
science or history related, and route accordingly.",
    handoffs=[science_manager, history_manager]
)

# Draw agent graph
draw_graph(triage_agent, filename="graph_visualization")
```


- The `draw_graph` function from the `agents.extensions.visualization` class takes as input any agent, draws the multi-agent system visualization, and saves it to your project's root folder as `graph_visualization.png`:



Guardrails

- **Guardrails** are a core SDK primitive used to **validate inputs and outputs** in agentic systems.
They act as a **protective safety layer**, ensuring system reliability and compliance.

Types of Guardrails

- **Input Guardrails**
 - Validate user input *before* it reaches the agent
- **Output Guardrails**
 - Validate agent responses *before* they reach the user

Guardrail Execution Pattern

1. Define a Guardrail Function

1. Returns a `GuardrailFunctionOutput`
2. Includes:
 1. `tripwire_triggered`: `Boolean`
 2. Context, agent, and input/output data

2. Evaluate the Tripwire Logic

1. Hardcoded checks
(e.g., *keyword detection*)
2. Agent-based reasoning
(*a separate agent decides whether to trigger*)

3. Handle the Tripwire Gracefully

1. Trigger a guardrail exception
2. Return a controlled, safe response to the user

Input guardrails

- Input guardrails act like a **flight attendant at the airport gate**
- They ensure **only valid and allowed requests** enter the agentic system
- Anything invalid is stopped *before* reaching the agent

Practical Use Case

- Applied to a **customer service agent**
- Guardrail checks user input *before* agent execution
- Example (naïve trigger):
 - If prompt contains the word “**complaint**”
 - → **Tripwire is triggered**
 - → Agent execution is stopped

input_guardrail.py

```
# Required imports
import os
from dotenv import load_dotenv
from agents import Agent, Runner, function_tool, trace
from agents import GuardrailFunctionOutput,
InputGuardrailTripwireTriggered, input_guardrail, RunContextWrapper,
TResponseInputItem

# Load environment variables from the .env file
load_dotenv()

# Access the API key
api_key = os.getenv("OPENAI_API_KEY")
```

```
# Create a tool
@function_tool()
def get_order_status(orderID: int) -> str:
    """
    Returns the order status given an order ID
    Args:
        orderID (int) - Order ID of the customer's order
    Returns:
        string - Status message of the customer's order
    """
    if orderID in (100, 101):
        return "Delivered"
    elif orderID in (200, 201):
        return "Delayed"
    elif orderID in (300, 301):
        return "Cancelled"
```

```
# Create a guardrail

def complaint_detector_guardrail(
    ctx: RunContextWrapper[None],
    agent: Agent,
    prompt: str | list[TResponseInputItem]
) -> GuardrailFunctionOutput:

    tripwire_triggered = False

    if "complaint" in prompt:
        tripwire_triggered = True

    return GuardrailFunctionOutput(
        output_info="The word Complaint has been detected",
        tripwire_triggered=tripwire_triggered,
    )
```

```
# Define an agent
agent = Agent(name="Customer service agent",
              instructions="You are an AI Agent that helps respond to
customer queries for a local paper company",
              model="gpt-4o",
              tools=[get_order_status],
              input_guardrails=[ complaint_detector_guardrail])
```

Finally, we wrap it all up with a simple loop to interact with the agent. The guardrail will check every input before passing it through to the agent:

```
with trace("Input Guardrails"):
    while True:
        question = input("You: ")
        result = Runner.run_sync(agent, question)
        print("Agent: ", result.final_output)
```


Let's try it out. Run the program, and enter the following message:

You: What's the status of my order? My order ID is 200

Agent: The status of your order with ID 200 is: Delayed. If you have any further questions or need assistance, please let me know!

The tripwire has not been triggered as the user prompt does not contain the word "complaint."

Let's try again and purposely trigger the tripwire:

You: I have a complaint

InputGuardrailTripwireTriggered error

Here, we can see that the system raises an **InputGuardrailTripwireTriggered** error and halts execution. Now, let's modify our script to more gracefully handle the tripwire exception. We can do this by adding the following to the end of the code:

Here, we can see that the system raises an `InputGuardrailTripwireTriggered` error and halts execution. Now, let's modify our script to more gracefully handle the tripwire exception. We can do this by adding the following to the end of the code:

```
...  
with trace("Input Guardrails"):  
    while True:  
        try:  
            question = input("You: ")  
            result = Runner.run_sync(agent, question)  
            print("Agent: ", result.final_output)  
        except InputGuardrailTripwireTriggered:  
            print ("The tripwire has been triggered. Please call us  
instead to register complaints.")
```

You: I have a complaint

The tripwire has been triggered. Please call us instead to register complaints.

Here, we can see that the tripwire is still triggered and our agent execution halts, but the exception is gracefully handled and a nice message is displayed to the user.

We can also see the input guardrail in the **Traces** module:

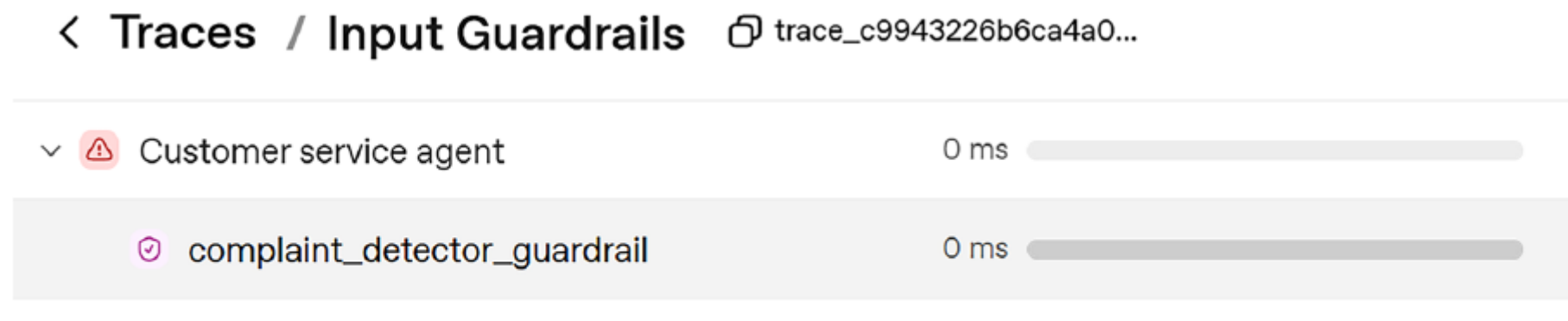


Figure 8.2: Input guardrail in Traces module

Smarter Input Guardrails Using an Agent

Problem with Basic Guardrails

- Simple keyword checks (e.g., detecting the word “*complaint*”) are fragile
- Users may phrase requests differently
- Irrelevant or misuse queries may bypass the guardrail

Improved Pattern: Agent-Based Guardrails

- Introduce a **separate lightweight agent**
- Runs on a **cheaper, simpler model**
- Sole responsibility:
 - Assess whether the user prompt is **relevant to customer service**
 - Decide whether to **trigger the guardrail tripwire**

input_guardrail_agent.py

```
# Required imports
import os
from dotenv import load_dotenv
from agents import Agent, Runner, function_tool, trace
from agents import GuardrailFunctionOutput,
InputGuardrailTripwireTriggered, input_guardrail, RunContextWrapper,
TResponseInputItem
from pydantic import BaseModel

# Load environment variables from the .env file
load_dotenv()

# Access the API key
api_key = os.getenv("OPENAI_API_KEY")
```

Next, we'll create a simple tool for checking the order status. This is the same as in our earlier examples, but now it will serve as part of our customer service workflow:

```
# Create a tool
@function_tool()
def get_order_status(orderID: int) -> str:
    """
    Returns the order status given an order ID
    Args:
        orderID (int) - Order ID of the customer's order
    Returns:
        string - Status message of the customer's order
    """
    if orderID in (100, 101):
        return "Delivered"
    elif orderID in (200, 201):
        return "Delayed"
    elif orderID in (300, 301):
        return "Cancelled"
```

Now, let's define a Pydantic model that the guardrail agent will use to indicate whether the prompt is relevant to customer service or not:

```
class GuardrailTrueFalse(BaseModel):  
    is_relevant_to_customer_service_orders: bool
```

With this model, we can now create the guardrail agent. Its only job is to decide whether the user prompt is relevant to customer service and order-related issues:

```
# Create a guardrail agent  
guardrail_agent = Agent(  
    name="Guardrail check",  
    instructions="You are an AI agent that checks if the user's prompt is  
relevant to answering customer service and order related questions",  
    output_type=GuardrailTrueFalse,  
)
```

Next comes the guardrail function itself. This function runs the guardrail agent and triggers the tripwire if the prompt is irrelevant:

```
# Create a guardrail

async def relevant_detector_guardrail(
    ctx: RunContextWrapper[None],
    agent: Agent,
    prompt: str | list[TResponseInputItem]
) -> GuardrailFunctionOutput:

    result = await Runner.run(guardrail_agent, input=prompt)

    tripwire_triggered = False

    if result.final_output.is_relevant_to_customer_service_orders ==
False:
        tripwire_triggered = True

    return GuardrailFunctionOutput(
        output_info="The word Complaint has been detected",
        tripwire_triggered=tripwire_triggered
    )
```


Finally, let's define our main customer service agent, attach the guardrail, and run it inside a simple loop:

```
# Define an agent
agent = Agent(name="Customer service agent",
              instructions="You are an AI Agent that helps respond to
customer queries for a local paper company",
              model="gpt-4o",
              tools=[get_order_status],
              input_guardrails=[relevant_detector_guardrail])

with trace("Input Guardrails"):
    while True:
        try:
            question = input("You: ")
            result = Runner.run_sync(agent, question)
            print("Agent: ", result.final_output)
        except InputGuardrailTripwireTriggered:
            print("This comment is irrelevant to customer service.")
```

You: What's the meaning of life?

This comment is irrelevant to customer service



Note

One small but important adjustment here is that the guardrail function is defined as `async`. This is necessary because the guardrail itself is invoking another agent asynchronously (`Runner.run`). In practice, this means you'll need to use asynchronous guardrail functions whenever they call out to other agents.

Output guardrails

What Are Output Guardrails?

- Validate **agent responses before reaching the user**
- Act as the **final checkpoint** in an agentic system

Why Output Guardrails Matter

- Enforce **format compliance**
- Prevent **hallucinations or irrelevant responses**
- Redact **sensitive or unsafe information**
- Ensure **policy and business rule adherence**

Customer Support Example

If the agent outputs something irrelevant (such as an apology without status details, or worse, a hallucination), we would want the system to intercept it before it reaches the customer.

Output guardrails

How Output Guardrails Work

1. Agent generates a response
2. Output guardrail function is invoked
3. Guardrail evaluates the response
4. Either Tripwire triggered → response blocked
Or Passes → response sent to user

Technical Notes (OpenAI Agents SDK)

- Implemented as a **guardrail function**
- Returns `GuardrailFunctionOutput`
- If `output_type` is defined:
 - Guardrail receives a **structured object**
 - Enables precise field-level validation

Example: guardrail_agent.py

```
# Required imports
```

```
import os
```

```
from dotenv import load_dotenv
```

```
from agents import Agent, Runner, function_tool, trace
```

```
from agents import GuardrailFunctionOutput,
```

```
OutputGuardrailTripwireTriggered, output_guardrail, RunContextWrapper
```

```
from pydantic import BaseModel
```

```
# Load environment variables from the .env file
```

```
load_dotenv()
```

```
# Access the API key
```

```
api_key = os.getenv("OPENAI_API_KEY")
```

Next, we define simple Pydantic models for our outputs:

```
class MessageOutput(BaseModel):  
    response: str  
  
class GuardrailTrueFalse(BaseModel):  
    is_relevant_to_customer_service: bool
```

This agent checks whether the main agent's response is valid for customer service:

```
# Create a guardrail agent  
guardrail_agent = Agent(  
    name="Guardrail check",  
    instructions="You are an AI agent that checks if the agent response is  
relevant to answering a customer service question and not hallucinating",  
    output_type=GuardrailTrueFalse  
)
```

We then write the function that enforces the guardrail logic:

```
# Create a guardrail
@output_guardrail
async def relevant_detector_guardrail(
    ctx: RunContextWrapper[None],
    agent: Agent,
    output: MessageOutput
) -> GuardrailFunctionOutput:

    result = await Runner.run(guardrail_agent, input=output)

    tripwire_triggered = False

    if result.final_output.is_relevant_to_customer_service == False:
        tripwire_triggered = True

    return GuardrailFunctionOutput(
        output_info="",
        tripwire_triggered=tripwire_triggered
    )
```


Here, we define a deliberately “hallucinating” customer service agent to show how the guardrail catches invalid output:

```
# Define an agent
agent = Agent(name="Customer service agent",
              instructions="You are an AI Agent that outputs random song  
lines and poems", # to force model to hallucinate and trigger the output  
guardrail
              output_guardrails=[relevant_detector_guardrail])
```

Finally, we run the agent in a loop and catch cases where the guardrail is triggered:

```
with trace("Output Guardrails"):
    while True:
        try:
            question = input("You: ")
            result = Runner.run_sync(agent, question)
            print("Agent: ", result.final_output)
        except OutputGuardrailTripwireTriggered:
            print("The agent system did not produce an output. Please try  
again")
```

If we run the program, it will trigger the output guardrail no matter what we type in as we have configured the agent to hallucinate purposely:

You: what's the status of my return?

The agent system did not produce an output. Please try again

HOW TO BUILD AI AGENTS

USE THIS STEP-BY-STEP FRAMEWORK

STEP 1

» SET THE AGENT'S PURPOSE AND RESULT

- What will the agent do?
- Who will it support?
- What kind of output should it create?

Example: A content research agent that reviews articles and summarizes key points.

STEP 2

» BUILD STRUCTURED INPUT AND OUTPUT

- Use Pydantic AI or JSON Schemas for inputs and outputs.
- Keep the format clean and API-like.

Tools: LangChain Output Parsers, Pydantic AI

STEP 3

» SHAPE AND TUNE THE AGENT'S BEHAVIOR

- Start with a clear role-based system prompt.
- Use Prefix Tuning or Prompt Tuning for consistent behavior.

Tools: Claude, GPT, Prompt Tuning

STEP 4

» ADD REASONING AND TOOL ACCESS

- Use ReAct or Chain-of-Thought for reasoning steps.
- Enable tools like search, code, or document lookup.

Tools: OpenAI Tools, LangChain, ReAct Framework

STEP 5

» ORGANIZE MULTI-AGENT ROLES (IF NEEDED)

- Use orchestration frameworks to define each agent's job.
- Create Planner, Researcher, and Writer agents with their own schemas.

Tools: LangGraph, CrewAI, OpenAI Swarm

STEP 6

» ADD MEMORY AND EXTENDED CONTEXT (RAG)

- Decide if the agent needs context from earlier work.
- Use summary, conversational, or vector memory.

Tools: Chroma, Zep, LangChain Memory

STEP 7

» ADD SPEECH OR VISION FEATURES (OPTIONAL)

- Use tools like Coqui or ElevenLabs for speech.
- Use vision models (GPT/LLaMA) for interpreting images.

This allows the agent to see and speak.

STEP 8

» FORMAT AND DELIVER THE OUTPUT

- Export results as Markdown, PDF, or JSON.
- Keep the output clean and predictable.

Tools: Pydantic AI, LangChain Output Parsers

STEP 9

» EMBED IN A UI OR API LAYER (OPTIONAL)

- Create a small UI or let the agent run through an API.
- Use Streamlit, Gradio, or FastAPI to make it accessible.

This turns the agent into a real product.

STEP 10

» TEST, REVIEW, AND IMPROVE

- Run test prompts to measure reliability.
- Use logs and benchmarks to improve performance.

Tools: Custom dashboards, MCP Logs, OpenAI Evaluation API